DAW · PROYECTO INTERMODULAR

SEGUNDA ENTREGA

Tupper planificación y tecnologías

Elección del stack, cronograma, diseño técnico (entidad-relación, arquitectura, flujo funcional) y mockups de las pantallas principales.

Tupper es el nombre de la aplicación; **Recetario** es el nombre de trabajo del proyecto durante su desarrollo.

Roberto de la Casa Sotillo
Ciclo Formativo de Grado Superior · DAW

Entrega del 8 de noviembre de 2026
github.com/metatron18es/recetario

01 Elección de tecnologías

Para que cualquier persona que lea esta memoria entienda las decisiones técnicas, conviene aclarar antes dos ideas: el **frontend** es la parte de la aplicación que el usuario ve y con la que interactúa en su navegador; el **backend** es la parte que funciona en un servidor, fuera de su vista, gestiona los datos y protege la información.

Frontend — Vue 3

Construye la interfaz visual. Sencillo de aprender, muy documentado, y las pantallas se actualizan solas cuando cambian los datos (el total del calendario se recalcula al añadir una receta, sin recargar la página).

Backend — Node.js + Express

Responde a las peticiones de la aplicación («dame mis recetas», «genera mi calendario»). Permite usar TypeScript tanto en frontend como en backend, simplificando el desarrollo.

TypeScript

Extensión de JavaScript que obliga a definir de antemano el tipo de cada dato. Evita errores típicos y el propio editor avisa antes de ejecutar nada.

Base de datos — PostgreSQL

Guarda de forma organizada y permanente usuarios, recetas, preferencias y calendarios. Gratuita, fiable y adecuada para datos relacionados entre sí.

Prisma (ORM)

Traductor entre el código y la base de datos: los datos se piden con código normal en vez de SQL, y avisa de errores de nombre o tipo antes de que el programa falle.

Seguridad — JWT y bcrypt

JWT es una «pulsera identificativa» digital tras iniciar sesión. bcrypt revuelve las contraseñas de forma irreversible antes de guardarlas en la base de datos.

Validación — Zod

Comprueba que los datos que llegan al servidor tienen el formato esperado antes de guardarlos, evitando datos incorrectos o malintencionados.

Pruebas — Vitest

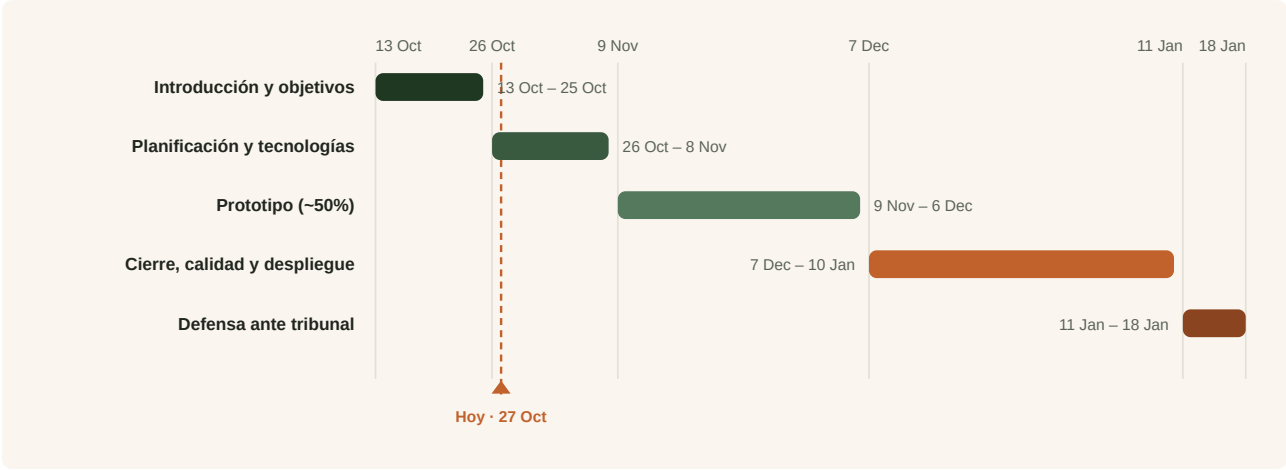
Programas que comprueban automáticamente que el código sigue funcionando como se espera, detectando errores antes de que lleguen a producción.

Despliegue — Vercel y Neon · Control de versiones — GitHub y GitHub Actions

Vercel aloja la parte visual y el servidor; Neon hace lo mismo con la base de datos: cada cambio subido se publica solo, sin servidor propio que mantener. GitHub guarda el historial de cambios y GitHub Actions ejecuta las pruebas automáticamente en cada subida, avisando si algo se ha roto.

02 Planificación

El proyecto se organiza en cinco fases con fecha de entrega fija marcada por el tutor, gestionadas mediante un tablero Kanban en GitHub (no sprints cerrados, al ser un desarrollo individual sin equipo con el que coordinar iteraciones).



CRONOGRAMA DEL PROYECTO · CINCO FASES · OCTUBRE 2026 – ENERO 2027

FASE	PERIODO	DURACIÓN	QUÉ INCLUYE
Introducción y objetivos	13–25 oct	13 días	Backlog de historias de usuario, elección del stack tecnológico, apartados 1-2 de la memoria
Planificación y tecnologías	26 oct – 8 nov	14 días	Diagrama entidad-relación, diagrama de arquitectura, diagrama de flujo funcional, cronograma, mockups, esqueleto desplegado
Prototipo (~50%)	9 nov – 6 dic	28 días	Autenticación, CRUD de recetas, preferencias nutricionales, motor de reglas, diseño del protocolo de pruebas
Cierre, calidad y despliegue	7 dic – 10 ene	35 días	Calendario semanal editable, tests automatizados, despliegue final en producción, resto de apartados de la memoria
Defensa ante tribunal	11–18 ene	8 días	Ensayo cronometrado de la exposición oral

El desglose completo en tareas, con las etiquetas de epic y prioridad, está en el tablero Kanban del proyecto y en los hitos del repositorio, donde cada fase es un hito con su fecha límite y las tareas (issues) asignadas.

03 Diagrama entidad-relación

El modelo de datos se organiza en tres bloques: identidad y preferencias del usuario, catálogo de recetas, y planificación semanal.

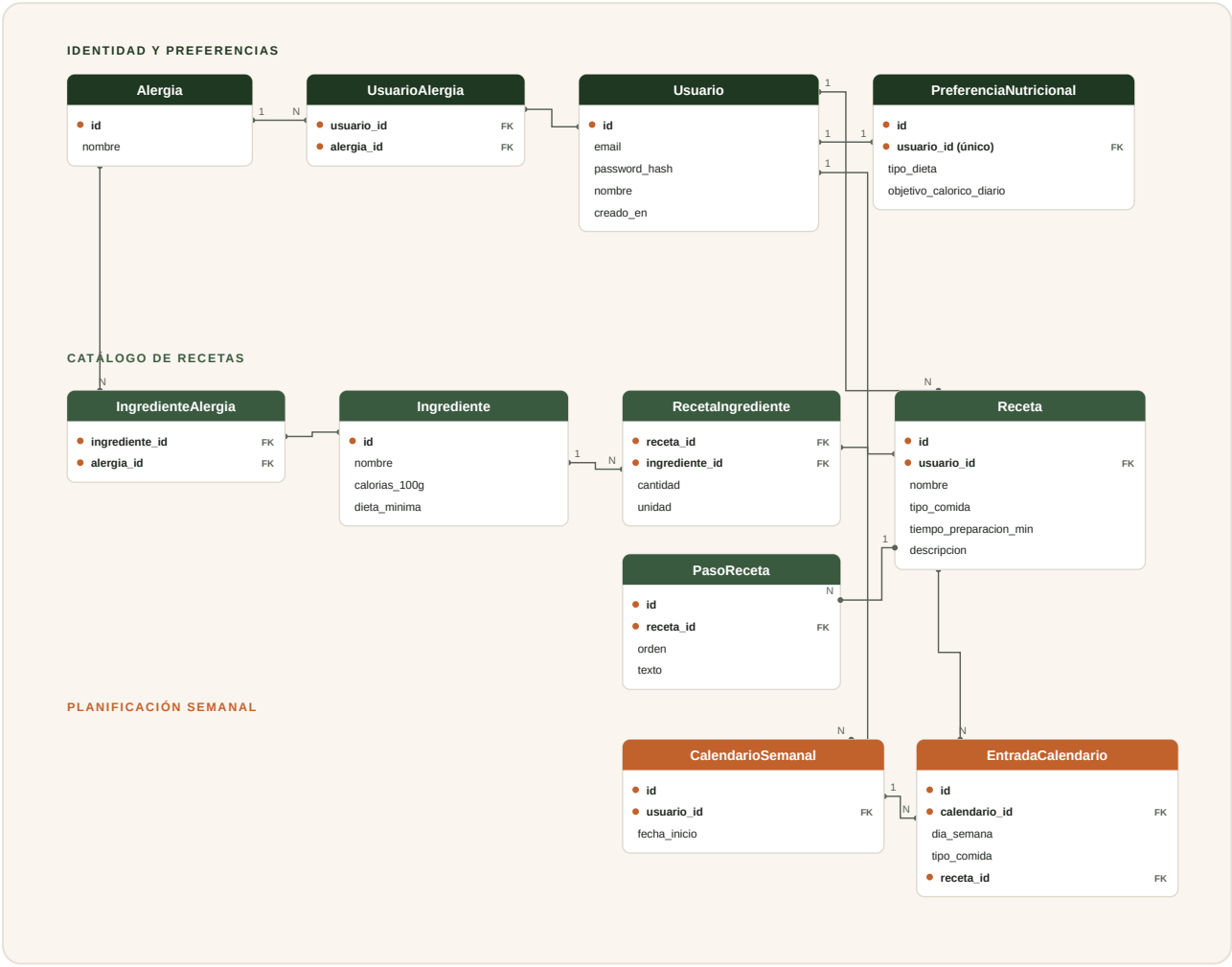


DIAGRAMA ENTIDAD-RELACIÓN · 11 ENTIDADES

Usuario — cuenta y autenticación

PreferenciaNutricional — dieta y objetivo calórico (1:1 con Usuario)

Alergia — catálogo de alergias/intolerancias

Ingrediente — catálogo con calorías por 100g y dieta mínima que lo admite

Receta — autoría, tipo de comida, tiempo y descripción

PasoReceta — pasos de preparación ordenados de cada receta

CalendarioSemanal — una semana por usuario

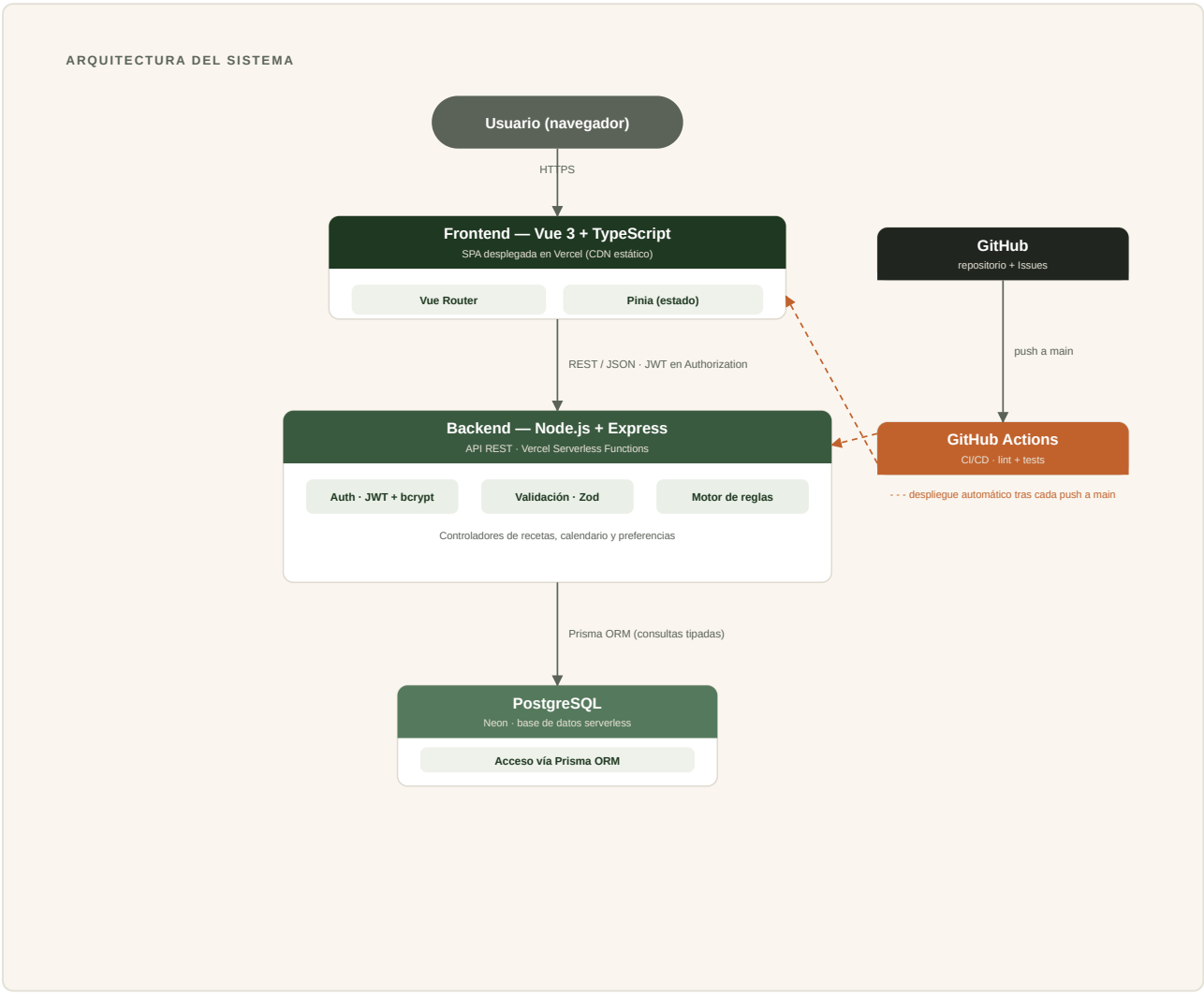
EntradaCalendario — celda día × tipo de comida

UsuarioAlergia / IngredienteAlergia — cruces N:M para el motor de reglas

RecetaIngrediente — cantidad y unidad por ingrediente

04 Diagrama de arquitectura

> Arquitectura en tres capas desacopladas, desplegadas por separado y conectadas mediante un pipeline de integración continua.

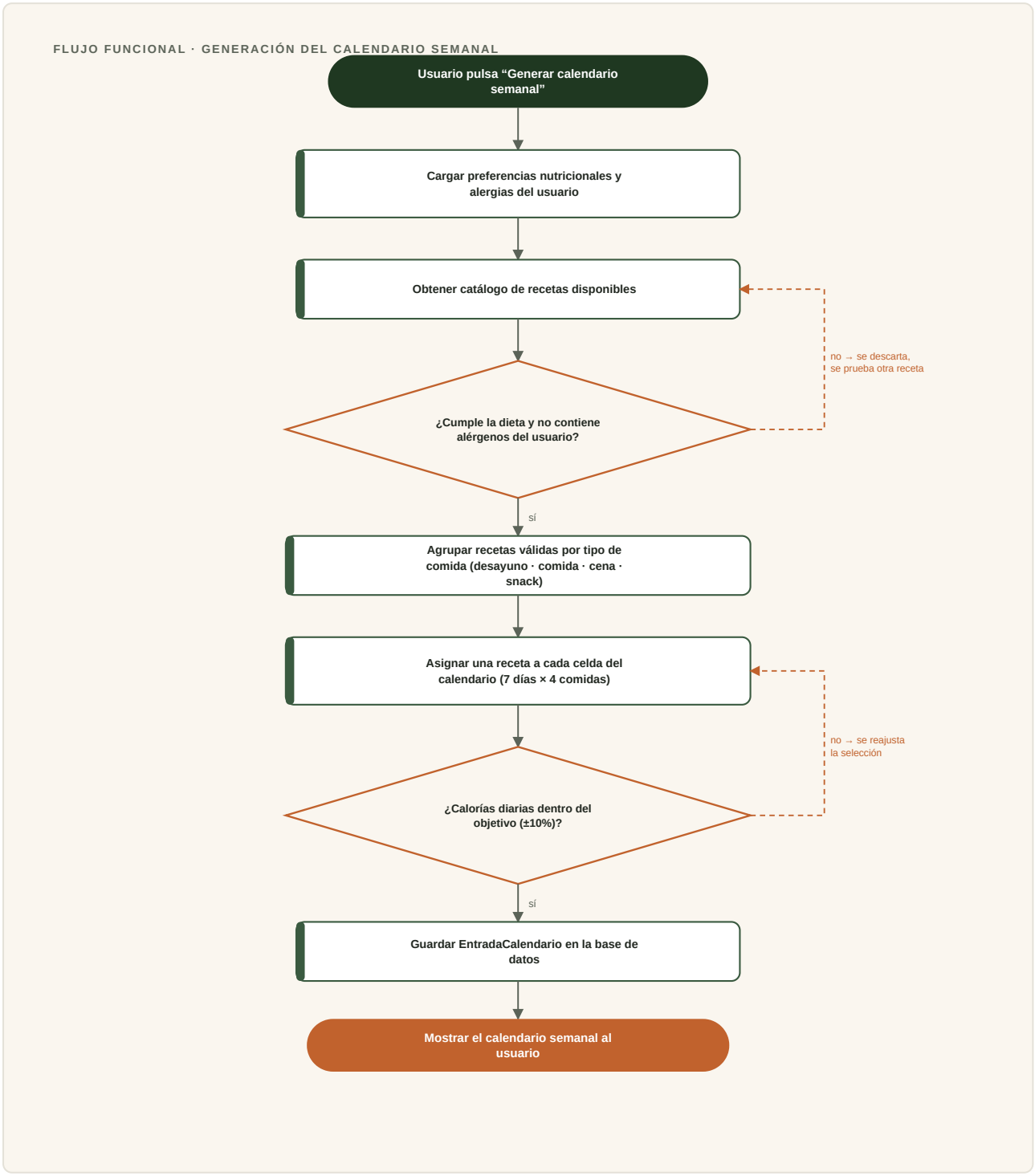


FRONTEND (VUE 3) · BACKEND (EXPRESS) · BASE DE DATOS (POSTGRESQL/NEON) · CI/CD (GITHUB ACTIONS)

El navegador consume una SPA en Vue 3 desplegada como contenido estático en Vercel. Esa SPA habla por HTTPS con una API REST en Node.js/Express (también en Vercel, como funciones serverless), que valida la identidad mediante JWT, los datos de entrada con Zod, y resuelve la lógica de negocio del motor de reglas. El acceso a PostgreSQL (alojado en Neon) se hace siempre a través de Prisma. Cada push a `main` dispara GitHub Actions, que ejecuta lint y tests antes de desplegar automáticamente frontend y backend.

05 Diagrama de flujo funcional

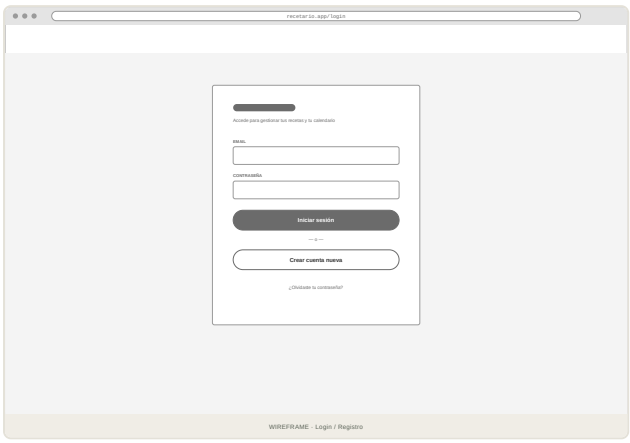
Describe cómo el motor de reglas genera el calendario semanal: filtra recetas por dieta y alergias, las asigna a cada celda, y ajusta el resultado hasta cumplir el objetivo calórico.



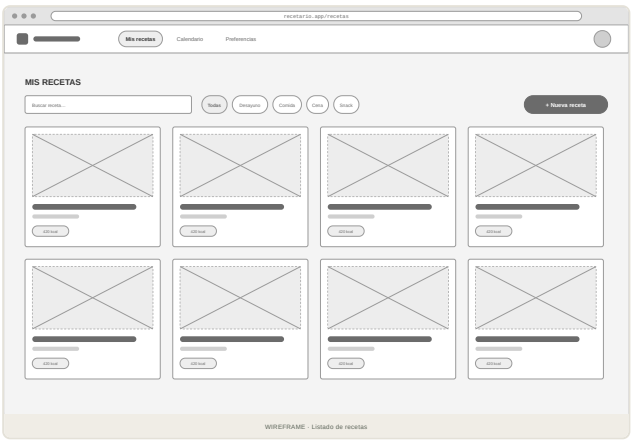
FLUJO · GENERACIÓN AUTOMÁTICA DEL CALENDARIO SEMANAL

06 Mockups de las pantallas principales

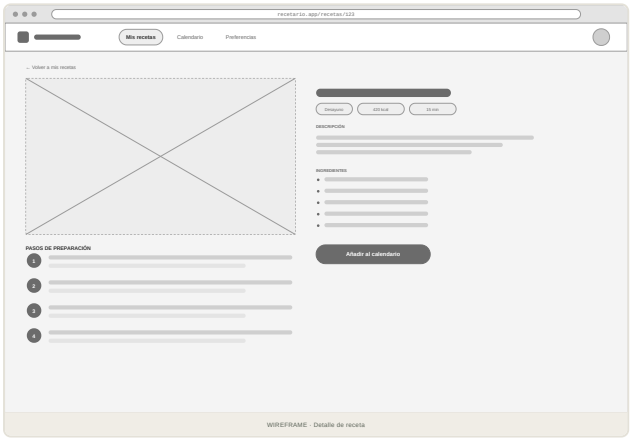
Wireframes de baja fidelidad: fijan la estructura y el contenido de cada pantalla sin entrar todavía en el diseño visual, que se abordará en una fase posterior.



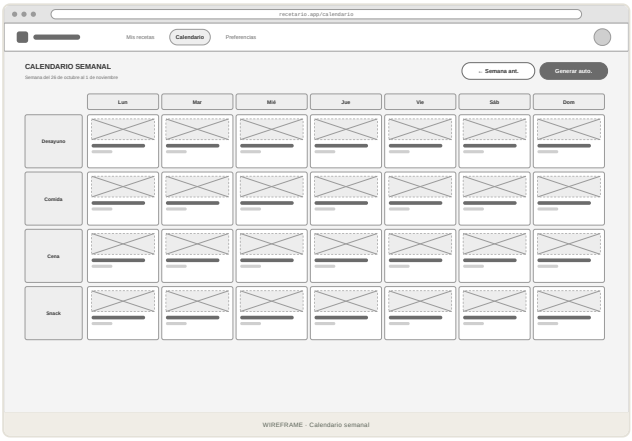
LOGIN / REGISTRO



LISTADO DE RECETAS



DETALLE DE RECETA



CALENDARIO SEMANAL

07 Control de versiones y estado del esqueleto

El código se versiona con Git y se aloja en un repositorio público de GitHub. Al ser un desarrollo individual, el flujo de trabajo es sencillo: cada historia de usuario se desarrolla en su propia rama (`feature/us-XX-descripcion`), con commits pequeños y descriptivos, y se integra en `main` mediante un pull request, lo que deja un historial organizado y revisable aunque no haya otra persona revisando.

Este flujo se aplica desde el 4 de noviembre de 2026, cuando se reinició el historial del repositorio: desde entonces `main` está protegida, no admite commits directos y todos los cambios entran mediante pull request con una plantilla común.

1 · Rama por historia

`feature/us-XX-descripcion` a partir de `main`, un issue por historia de usuario.

2 · Commits y PR

Commits pequeños y descriptivos; integración en `main` mediante pull request, con plantilla de PR y `Closes #N` para cerrar la issue.

3 · CI automática

GitHub Actions construye y prueba backend y frontend en cada pull request hacia `main` y en cada subida a `main` (issue #21).

4 · Tablero Kanban

Backlog → Lista para hacer → En progreso → En revisión/test → Hecho.

Esqueleto desplegado. El proyecto Vue 3 y el proyecto Express están conectados entre sí y publicados en Vercel, con PostgreSQL en Neon y las migraciones aplicadas; la página de inicio comprueba la conexión con la API. El desarrollo real (autenticación, CRUD de recetas, motor de reglas) arranca en la fase «Prototipo (~50%)», del 9 de noviembre al 6 de diciembre.